

Programmazione 1

Corso c

>Correttezza di una
sequenza di assegnazioni

Alessandro Mazzei

Dipartimento di Informatica

Università degli Studi di Torino



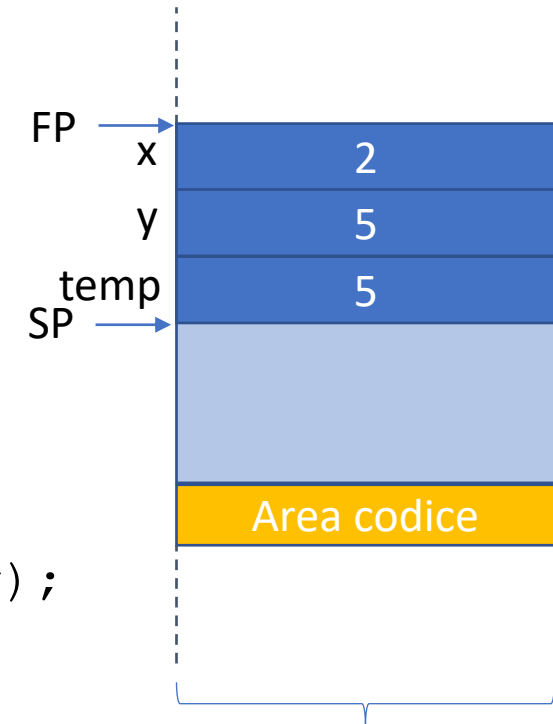
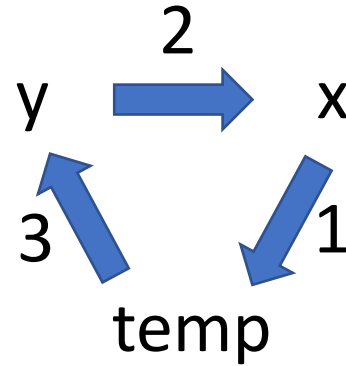
UNIVERSITÀ
DI TORINO

Correttezza

- Fino ad ora abbiamo visto degli algoritmi molto semplici basati su *sequenze di assegnazioni*, come lo **swap** di due valori con un buffer
- Essendo semplici, *ad occhio*, siamo certi che questi algoritmi sono **corretti** ovvero che producono sempre un output corretto su qualsiasi input

SWAP

```
int main(void) {  
    int x = 5;  
    int y = 2;  
    int temp;  
  
    temp = x;  
    x = y;  
    y = temp;  
    printf ("Le var x, y e valgono %d, %d\n", x, y);  
}
```



```
$ ./swap_2  
'Le var x, y e valgono 2, 5
```

Correttezza

- Nelle prossime lezioni le cose si complicheranno, quando introdurremo anche i concetti (e i costrutti) di *scelta* e *ripetizione*, necessari, insieme alla *sequenza*, per implementare un qualsiasi algoritmo (teorema di Bohm-Jacopini)
- Quando i programmi sono complessi è difficile (impossibile!) essere certi ad occhio della correttezza di un algoritmo

Correttezza formale

- Si parla di correttezza formale quando riusciamo a **dimostrare** che un algoritmo è corretto per ogni input **possibile** (ogni input che *garantisce* certe specifiche proprietà).
- Noi lo vedremo nel caso più semplice ovvero con algoritmi che usano solo assegnazioni, ma in linea di principio è possibile ampliare il formalismo a qualsiasi algoritmo
- Quindi, in questa lezione, algoritmo o programma sono sinonimi di *sequenza di assegnazioni*

Obiettivo: SWAP2

- Vogliamo dimostrare che questo algoritmo:

```
{  
  int a = 1;  
  int b = 3;  
  
  a = a + b;  
  b = a - b;  
  a = a - b;  
} //a=? b=?
```

Inverte correttamente i valori iniziali delle variabili intere a e b.

N.B niente terza variabile!

<assert.h>

- Intruciamo la macro **assert**, ovvero una funzione (che lavora in *preprocessing*) che controlla la verità di una condizione booleana:
 - Se **true**, va avanti senza far nulla
 - Se **false**, interrompe un messaggio, restituendo delle informazioni

```
{  
    int a = 1;  
    int b = 3;  
    assert (a+b!=4) ;  
}
```

```
$ ./a.out2  
Assertion failed: (a+b!=4), function main, file test.c, line 8.  
Abort trap: 6
```

- no debugging? -> NDEBUG

#define

- Introduciamo **#define**, ovvero un'istruzione che ci consente di definire le costanti in pre-processing

```
# define X 3
{
    int a = X;
    assert (a==3) ;
}
```


Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Semantica assiomatica: stato di un programma

- Introduciamo il concetto di **stato di un programma**, che consiste nel verificare l'uguaglianza dei **valori delle variabili** in un certo punto nell'esecuzione del programma
- Per farlo useremo delle **espressioni booleane** (atomiche o complesse) sui valori assunti dalle variabili
- In logica si chiamano **predicati** (*logica dei predicati*)

Esempio: stati 1, 2 e 3

```
{  
  int a = 3;  
  // (1) stato descritto col predicato a==3  
  int b = 12;  
  // (2) stato descritto col predicato a==3 && b==12  
  int y = a + b;  
  // (3) stato descritto col predicato a==3 && b==12 && y==15  
}
```

Esempio: stati 1, 2 e 3

```
{  
    int a = 3;  
    // (1) stato descritto col predicato a==3  
    assert(a == 3);  
    int b = 12;  
    // (2) stato descritto col predicato a==3 && b==12  
    assert(a == 3 && b == 12);  
    int y = a + b;  
    // (3) stato descritto col predicato a==3 && b==12 && y==15  
    assert((a == 3 && b == 12) && (y == 15));  
}
```

Logica booleana

- Stiamo usando la logica booleana (operatori **&&**, **||**, **!**) sulle uguaglianze, ovvero delle espressioni complesse che possono essere vere o false sulla base della verità delle singole uguaglianze
- **Tavole di verità:**

E'	E''	E = E' && E''
0	0	0
0	1	0
1	0	0
1	1	1

E'	E''	E = E' E''
0	0	0
0	1	1
1	0	1
1	1	1

E'	E
0	1
1	0

Logica booleana

• Stiamo usando la logica booleana (operatori &&, ||, !) sulle uguaglianze, ovvero delle espressioni complesse che possono essere vere o false sulla base della verità delle singole uguaglianze

IMPORTANTE: la logica booleana gioca un ruolo fondamentale nei costrutti di scelta e ripetizione!!!

- **Tavole di verità:**

E'	E''	E = E' && E''
0	0	0
0	1	0
1	0	0
1	1	1

E'	E''	E = E' E''
0	0	0
0	1	1
1	0	1
1	1	1

E'	E
0	1
1	0

Stato Iniziale e Stato Finale

- Lo **Stato Iniziale** è descritto tramite un predicato che fissa valori e relazioni dell'insieme di variabili **all'inizio** di un qualsiasi algoritmo
- Lo **Stato Finale** è descritto tramite un predicato che fissa valori e relazioni dell'insieme di variabili **al termine** di un qualsiasi algoritmo

```
// (0) stato iniziale
{
  // (1)
  a = x + 3;
  // (2)
  y = a + b;
}
// (3) stato finale
```


Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Pre-condizione e post-condizione di un'istruzione

Si definisce di **Pre-condizione** (**Post-condizione**) di un'istruzione il predicato che si trova **prima** (**dopo**) l'istruzione.

```
{  
  // (1) P1  
  a = x + 3; // I1  
  // (2) P2  
  y = a + b; // I2  
  // (3) P3  
}
```

- P1 è pre-condizione di I1
- P2 è pre-condizione di I2 e post-condizione di I1
- P3 è post-condizione di I2

Pre-condizione del programma

- Una **pre-condizione del programma** è un predicato che si può assumere essere vero all'inizio del programma

```
// (PRE) Pre-condizione: 10 <= x && (b == 32 - x || b == 16)
// (0) stato iniziale
{
  // (1)
  a = x + 3;
  // (2)
  y = a + b;
  // (3) stato finale
}
```

- N.B. prima di // (0)

Post-condizione del programma

- Una **post-condizione del programma** è un predicato che si può assumere essere vero alla fine del programma

```
// (PRE) Pre-condizione: 10 <= x && (b == 32 - x || b == 16)
// (0) stato iniziale
{
  // (1)
  a = x + 3;
  // (2)
  y = a + b;
  // (3) stato finale
}
// (POST) Post-condizione: 25 <= y
```

- N.B. dopo di // (3)

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Predicati su stati: perché?

- I predicati logici ci permettono di **dimostrare** una **tesi** a partire da una **ipotesi** mediante *inferenza logica*, ovvero una forma di calcolo che si basa sulle tavole di verità degli operatori booleani e sull'implicazione →
- Quindi possiamo partire dalla verità di un predicato P1 su uno stato (**ipotesi**) per verificare la verità di un predicato P2 un altro stato (**tesi**).
- Usando la logica, possiamo quindi rispondere a diversi tipi di domande sugli stati di un programma

Alcune domande tipiche sulla correttezza che posso risolvere con la LdF

1. Pre $\rightarrow$ Post?

2. Supponiamo che in `//(3)` valga il predicato $a < b$, quale deve essere il predicato più generale possibile in `// (0)` affinché ciò avvenga?

3. Supponiamo che in `//(1)` valga il predicato $12 \leq x \ \&\& \ a \% 2 == 0$, quali stati possiamo raggiungere al punto (2)?

```
// (PRE)
// (0) stato iniziale
{
    // (1)
    a = x + 3;
    // (2)
    y = a + b;
}
// (3) stato finale
// (POST)
```

Alcune domande tipiche sulla correttezza che posso risolvere con la LdF

1. Pre $\rightarrow$ Post?

2. Supponiamo che in `//(3)` valga il predicato $a < b$, quale deve essere il predicato più generale possibile in `// (0)` affinché ciò avvenga?
3. Supponiamo che in `//(1)` valga il predicato $12 \leq x \ \&\& \ a \% 2 == 0$, quali stati possiamo raggiungere al punto (2)?

```
// (PRE)
// (0) stato iniziale
{
    // (1)
    a = x + 3;
    // (2)
    y = a + b;
}
// (3) stato finale
// (POST)
```


Logica di Floyd (LdF): $\text{Pre} \rightarrow \text{Post}$

La Logica di Floyd (LdF) permette di stabilire se l'implicazione logica $\text{Pre} \rightarrow \text{Post}$ è vera, col seguente procedimento:

1. si verifica che l'implicazione $\text{Pre} \rightarrow (0)$ è vera, quando Pre è vero. Quindi anche (0) è necessariamente vero;
2. Si **dimostra** che l'implicazione $(i) \rightarrow (i+1)$ è vera, *usando* l'istruzione Istr_i
3. Si dimostra che l'implicazione $(N) \rightarrow (\text{Post})$ è vera.

Esempio di domanda di tipo 1 (Pre → Post)

Pre → Post?

- N.B già ci sono i **predicati sugli stati che ci interessano** ovvero PRE e POST!
- Possiamo ragionare in avanti (**forward**, parto da PRE) o all'indietro (**backward**, parto da POST)

```
// (PRE) 10 <= x
// (0) stato iniziale
{
  // (1) ??
  int a = x + 3;
  // (2) ??
  int b = 12;
  // (3) ??
  y = a + b;
  // (4) ??
}
// (POST) 25 <= y
```

Esempio di domanda di tipo 1

- Pre $\rightarrow$ Post?

- N.B già ci sono i predicati sugli stati che ci

IMPORTANTE: nel seguito useremo solo il ragionamento all'indietro (backward) per semplicità.

- Possiamo ragionare in avanti (forward,

parto da PRE) o all'indietro (backward,

parto da POST)

```
//PRE: 10 <= x
// (0) stato iniziale
{
// (1) ??
    int a = x + 3;
// (2) ??
    int b = 12;
// (3) ??
    y = a + b;
// (4) ??
}
//POST: 25 <= y
```

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Esempio: Pre→Post LdF con ragionamento *backward*

Ragioniamo indietro su *quello di cui abbiamo bisogno*:
**costruiamo il predicato *precedente* come quello più generale
per rendere vero il seguente dopo l'istruzione**

```
/* pre-condizione: 13 <= x post-condizione: 25 <= y */  
// (0) 10 <= x (stato iniziale)  
{ // (1) 25 <= (x + 3) + 12 (stato_1)  
  int a = x + 3;  
  // (2) 25 <= a + 12 (stato_2)  
  int b = 12;  
  // (3) 25 <= a + b (stato_3)  
  y = a + b;  
  // (4) 25 <= y (stato_4 finale)  
}
```



Esempio: Pre→Post LdF con ragionamento *backward*

- Vogliamo che Post sia vera, quindi, $25 \leq y$ deve essere vero in //(4) che è lo stato finale
- In //(4) deve valere $25 \leq y$. Inoltre, l'istruzione precedente è $y = a + b$. Quindi, in //(3) deve valere $25 \leq a + b$, poiché y non ha ancora sostituito l'espressione $a + b$;
- In //(3) deve valere $25 \leq a + b$. Inoltre, b è il nome del valore numerico 12 stabilito da $b = 12$. Quindi, in //(2) deve valere $25 \leq a + 12$ in cui b non ha ancora sostituito il valore 12;
- In //(2) deve valere $25 \leq a + 12$. Inoltre, a è il nome dell'espressione $x + 3$ stabilito da $a = x + 3$. Quindi, in //(1) deve valere $25 \leq x + 3 + 12$, in cui a non ha ancora sostituito l'espressione $x + 3$.
- In //(1) deve valere $25 \leq x + 15$, quindi $10 \leq x$.
- Tra //(1) e //(0) non ci sono istruzioni, quindi lo stato iniziale in //(0) deve coincidere con $10 \leq x$, predicato in //(1).

```
/* pre-condizione: 13 <= x post-  
condizione: 25 <= y */  
// (0) 10 <= x (stato iniziale)  
{ // (1) 25 <= (x + 3) + 12  
  (stato_1)  
    int a = x + 3;  
    // (2) 25 <= a + 12 (stato_2)  
    int b = 12;  
    // (3) 25 <= a + b (stato_3)  
    y = a + b;  
    // (4) 25 <= y (stato_4 finale)  
}
```

LdF backward: procedimento

1. decorare ogni punto del programma con un predicato;
2. per ogni $Istr_i$ verificare che, se il predicato P_{i+1} dopo $Istr_i$ è vero, allora è vera anche il predicato P_i prima di $Istr_i$;
3. verificare che se la pre-condizione è vera, allora è vero il predicato che descrive lo stato iniziale ($PRE \rightarrow stato_{iniziale}$);
4. verificare che se il predicato che descrive lo stato finale è vero, allora è vera la post-condizione ($stato_{finale} \rightarrow POST$).

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Formalizziamo il processo che porta da P2 a P1

```
{  
    // (1) P1  
    x = 3;  
    // (2) P2  
}
```

Negli esempi precedenti abbiamo ottenuto P2 da P1 (backward), in maniera *informale*. Ora vediamo formalmente e sistematicamente come fare.

Weakest Pre-condition

Quando ragioniamo backward, voglio calcolare $P1$ che sia più generale possibile affinché dopo l'istruzione di assegnamento valga $P2$: **Weakest Pre-condition**

Calcolo la Weakest Pre-condition

```
{  
  // (1) P1 == P2 [x/E1]  
  x = E1;  
  // (2) P2  
}
```



Calcolo backward: il predicato P1 si ottiene sostituendo in P2 tutte le occorrenze di x con E1.

N.B. : ... sostituendo in P2 tutte le occorrenze di x con E1: $P2 [x/E1]$

Calcolo la Weakest Pre-condition

```
{  
  // (1) 25 <= a+b  
  y = a+b;  
  // (2) 25 <= y  
}
```



- Trovo (1) sostituendo in (2) tutte le occorrenze di y con $a+b$.

Calcolo la Weakest Pre-condition

```
{  
  // (1)  $x == 8 \ \&\& \ 2 * x == 16$   
   $y = 2 * x;$   
  // (2)  $x == 8 \ \&\& \ y == 16$   
}
```



- Trovo (1) sostituendo in (2) tutte le occorrenze di y con $2 * x$.

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Correttezza di SWAP2 mediante calcolo della weakest precondition

```
//PRE a==m && b==n
{
  //(1) ??
  a = a + b;
  //(2) ??
  b = a - b;
  //(3) ??
  a = a - b; //E1
  //(4) stato_finale
}
//POST a==n && b==m
```

Correttezza di SWAP2 mediante calcolo della weakest precondition

```
//PRE a==m && b==n
{
  // (1) a==m && b==n
  // (1) b==n && a==m
  // (1) b==n && (a+b)-b==m
  a = a + b;
  // (2) b==n && a-b==m
  // (2) a-(a - b)==n && a-b==m
  b = a - b;
  // (3) a-b==n && b==m
  a = a - b; //E1
  // (4) a==n && b==m
}
//POST a==n && b==m
```


Correttezza di SWAP mediante calcolo della weakest precondition

```
//PRE a==x && b==y
{
  //(1) ??
  tmp = a;
  //(2) ??
  a = b;
  //(3) ??
  b = tmp
  //(4) stato_finale
}
//POST a==y && b==x
```

Outline

- Stati di un programma e predicati
- Pre-condizione e Post-condizione
- Domande sulla correttezza
- Logica di Floyd
 - Backward reasoning
 - Weakest Precondition
- SWAP2
- Esempi di esercizi d'esame

Ex. 5.9.2

```
//PRE (X)
{
    //(1)
    y = 2 * y;
    //(2)
    x = y/2 + 1;
    //(3)
    x = x + 1; //(4)
    y=2*x ;
    assert(x == 8 && y == 16); //POST
}
```

Modificare il codice come segue:

- nei punti //(2), //(3) e //(4)
specificare `assert` che servono per stabilire la weakest pre-condition dell'intero algoritmo al punto //(1);
- sostituire //(X) con un'assegnazione che assicura la verità della weakest pre-condition al punto //(1);
- aggiungere quanto necessario ad ottenere un sorgente C che non produce errori di alcun tipo.

```

#include <assert.h>
void main(void)
{
    int x, y, z ;
    y = 6 ; // (X) sostituito
    assert(y == 6); // weakest pre-condition
    assert((y == 6) && (2 * y == 12)); // semplificato
    assert((2 * y / 2 == 6) && (2 * (2 * y / 2 + 1) == 14));
    y = 2 * y ;
    assert((y / 2 == 6) && (2 * (y / 2 + 1) == 14));
    assert((y / 2 + 1 == 7) && (2 * (y / 2 + 1) == 14));
    x = y / 2 + 1;
    assert((x == 7) && (2 * x == 14)); // semplificato
    assert((x + 1 == 8) && (2 * (x + 1) == 16));
    x = x + 1 ;
    assert((x == 8) && (2 * x == 16));
    y = 2 * x ;
    assert(x == 8 && y == 16); //lo stato finale è anche la post-
condizione
}

```

Ex. 5.9.3

```
//pre-condizione:  Z == 0
//post-condizione: y == 8
{
// (0) stato iniziale
// (1)
int x = Z + 2;
int y = x + W;
}
```

Modificare il codice come segue:

- In //(1) scrivere la weakest pre-condition.
- Stabilire uno stato iniziale e completare la pre-condizione, aggiungendo un congiunto al predicato $Z == 0$, affinché la post-condizione sia vera.

```

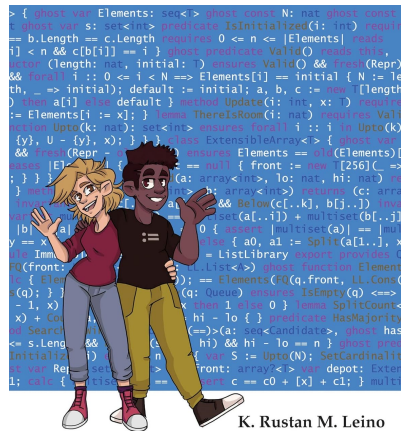
#include <assert.h>
#define Z 0
#define W 6
void main(void)
/*  La pre-condizione originale 'Z == 0'
    era insufficiente
    Pre-condizione completa : Z == 0 && W == 6
    Post-condizione immutata: y == 8
*/
{
    /* (0)  Affinché la weakest pre-condition
            sia vera, occorre che nello stato
            iniziale sia vero anche W == 6.
            Quindi la pre-condizione deve
            imporre che W == 6.

    */
    assert(Z + W == 6); /*  semplificato è la weakest-precondition  */
    assert(Z + 2 + W == 8);
    int x = Z + 2;
    assert(x + W == 8);
    int y = x + W;
    assert(y == 8);      // post-condizione
}

```

Riferimenti bibliografici

- Capitolo 5 Indice_Lezioni del prof. Roversi
- https://it.wikipedia.org/wiki/Logica_di_Hoare
 - Sezioni: Tripla di Hoare, Assioma dell'assegnamento, Regola della conseguenza.
- Cap. 2 di Leino&Leino. **Program Proofs**. MIT Press, 2023.



K. Rustan M. Leino
illustrated by Kaleb Leino

PROGRAM PROOFS

